Documentation File

Raşa Mecbur

(220911714)

- **Project Overview:**
  I built this chat application so multiple people can talk to each other through a central server. Multiple clients can connect to a single server and communicate in real time.

  -*Technologies used:*

  -Java Socket programming (java.net.Socket, ServerSocket).

  -Java Multithreading (Thread, Runnable)

  - Java Swing GUI (JFrame, JTextArea, JList, etc.)

  -Producer-Consumer Design Pattern.

  -Synchronization (synchronized, wait(), notifyAll())

- **How to Run the Application:**

-*Step 1: Start the Server*

. Open Source Code/src/server/ChatServer.java
. Run the file
. Enter Port Number
. Click “Start Server” button.
. You will see: “Server started on port 12345”.

-*Step 2: Start the Client*

. Go to Source Code/src/client/LoginGUI.java
. Run the file (click the run button)
. Enter connection details:
- Server IP: localhost (if same computer) or (IP address).
- Port: 12345
- Username: [any name can be used]
- Click “Login”.
- Chat window will open automatically.

-*Step 3: Add more Users*

. Run LoginGUI.java again for each additional user.

. Use different usernames.

. All users can chat with each other.

-*Step 4: Admin Controls (Server Window)*

. View active users list.

. Select a user and click “Kick selected User” to remove them.

. Click “Stop Server” to shut down.

- **Code Structure Explanation:**

  Server-Side Files(6 files):

  1. Request.java

  - Data structure for all client request
  - Contains Type enum (LOGIN, MESSAGE, LOGOUT, KICK, STOP_SERVER)
  - Stores username, message content, target user

2. SharedBuffer.java

- Implements the bounded buffer (size = 100)
- Uses synchronized, wait(), notifyAll() for thread safety
- put() method adds requests (called by producers)
- take() method removes requests (called by consumer)

3. ClientHandler.java (Producer Thread)

- One thread per connected client
- Listens for messages from client
- Creates Request objects and puts them in SharedBuffer
- Does NOT process requests directly

4. Dispatcher.java (Consumer Thread)

- Single thread that takes requests from SharedBuffer
- Creates a new Worker thread for each reques
- Implements the Consumer role

5. Worker.java (Request Processor)

- New thread created for each request
- Processes the request (login, message, logout, kick)
- Updates shared data and broadcasts when needed
- Thread dies after processing

6. ChatServer.java

- Main server class with Swing GUI
- Creates ServerSocket to listen for connections
- Starts Dispatcher thread
- Holds shared data (activeUsers, clients lists)
- Provides broadcast methods to all clients

Client-Side Files (3 files):

7. ChatClient.java

- Handles all network communication
- Connects to server using Socket
- Sends login, message, logout requests
- Listens for incoming messages from server

8. LoginGUI.java (Entry Point)

- Login screen with Server IP, Port, Username fields
- Creates ChatClient and attempts connection
- Opens ChatGUI on successful login

9. ChatGUI.java

- Main chat interface
- Displays messages and online users
- Has text field and send button for messages

- **Request Types and Handling:**

Request Type	Trigger	Server Action
LOGIN	User clicks login	Add to activeUsers, broadcast "joined"
MESSAGE	User sends chat	Broadcast message to all clients

LOGOUT	User clicks Exit	Remove from list, broadcast "left"
KICK	Admin clicks Kick	Disconnect user, broadcast "removed"
STOP_SERVER	Admin clicks Stop	Disconnect all, shutdown

- **Producer-Consumer Design**

  This project implements the required Producer-Consumer pattern:

  - *Producers: ClientHandler Threads.*

    . ClientHandler threads

    . Each client has its own ClientHandler thread

    . They receive requests from clients

    . They INSERT requests into the SharedBuffer

    . They do NOT process requests

  -*Shared Buffer: SharedBuffer class*

    . Fixed size queue (capacity = 100)

    . All producers share ONE buffer (not separate buffers)

    . Uses wait() when full, notifyAll() when space available

  -*Consumer: Dispatcher thread*

    .Single thread that REMOVES requests from buffer

    .Creates a Worker thread for each request

    . Immediately goes back to get next request

  -*Workers: Worker threads*

    . Each request gets its own Worker thread

    .Worker actually PROCESSES the request

    .Worker dies after processing

This design decouples request receipt from request processing.

- **Synchronization Implementation**

  All shared data is properly synchronized:

  1. SharedBuffer:
     - synchronized methods for **put()** and **take()**
     - **wait()** when buffer is full or empty
     - **notifyAll()** to wake waiting threads

  2. Active Users List:
     - synchronized blocks around all access
     - Prevents race conditions when adding/removing users

  3. Client Connections List:
     - synchronized blocks when iterating or modifying
     - Prevents ConcurrentModificationException

4. Broadcast Operations:

- Synchronized on clients list while sending messages

- **Implementation Notes**

1. The server MUST be started before any clients
2. Each client needs a unique username
3. The server window does NOT auto-popup - check it manually
4. Port 12345 is default, can be changed
5. Use "localhost" for same-computer testing
6. The bounded buffer size is 100 (configurable)

- **Troubleshooting**

1- Problem: "Failed to connect to server"

Solution: Make sure server is running AND "Start Server" is clicked

2- Problem: Login window doesn't close

Solution: Server didn't send LOGIN_SUCCESS - check server is running

3- Problem: Messages not appearing

Solution: Check if all clients are connected to same IP and port

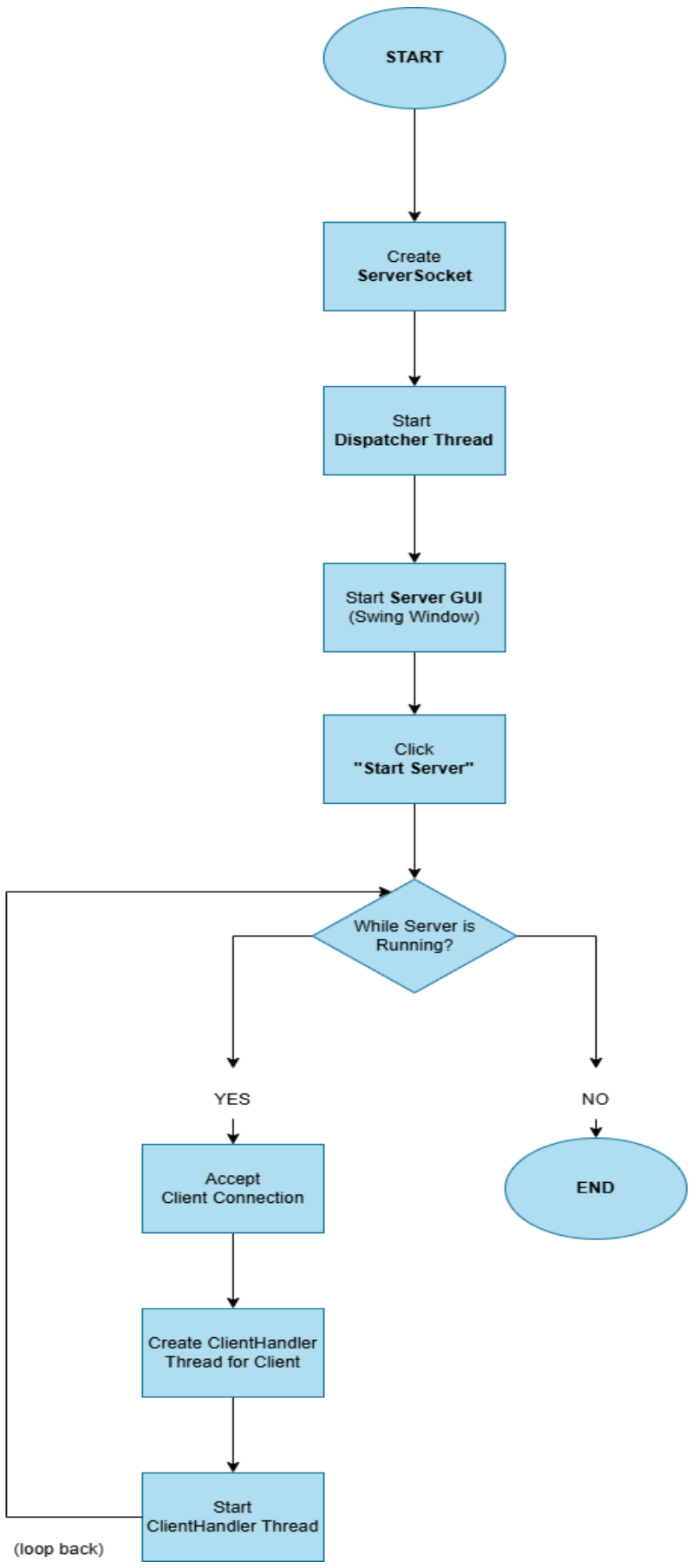
- **CONCLUSION**

This project successfully implements all requirements:

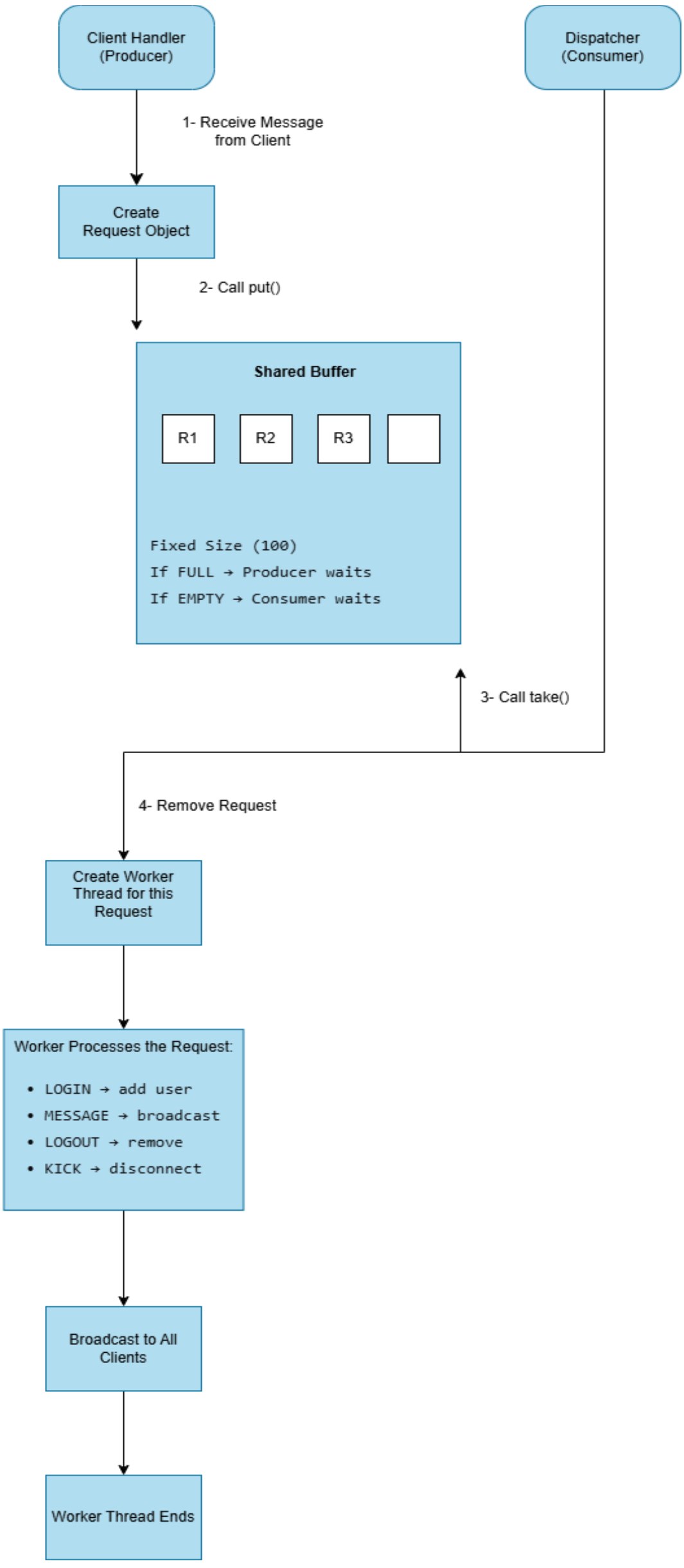
- Java Socket Programming
- Multithreading
- Producer-Consumer Pattern with Shared Bounded Buffer
- Synchronization with wait/notify
- Java Swing GUI for both server and client
- All 5 request types (Login, Message, Logout, Kick, Stop)
- The trickiest part was getting the bounded buffer to work with wait() and notifyAll(). After debugging, I finally understood how producers and consumers communicate through the shared buffer.

- **FlowCharts**

Server Flow Chart: Shows how the server starts and accepts clients



Producer – Consumer Flow Chart



Client Flow Chart